

클라우드 기초 개념과 AWS 환경 세팅



학습 목표

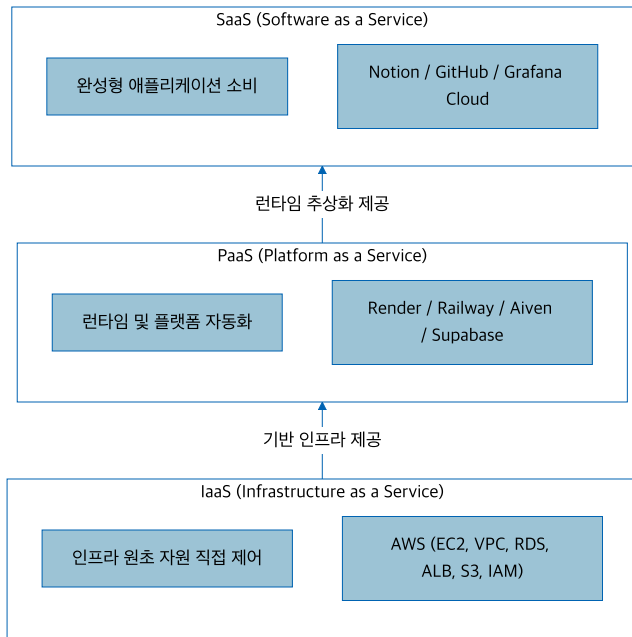
- 클라우드 서비스 3대 모델(**IaaS** · **PaaS** · **SaaS**)의 관리 책임 경계를 구분하고 대표 서비스를 분류합니다
- IAM Identity Center 포털에서 비밀번호를 변경하고 MFA를 등록해 안전한 인증 체계를 갖추습니다
- AWS CLI v2에 SSO 프로필을 연동해 자동 갱신되는 단기 자격 증명으로 클라우드 자원을 제어합니다
- 기본 VPC에 보안 그룹과 SSH 키 페어를 만들고 **t4g.nano** 인스턴스를 프로비저닝한 뒤 중지까지 연결합니다

클라우드는 인프라 자원을 직접 구매하지 않고 인터넷을 통해 임대하는 운영 패러다임입니다. 본 차시는 웹 콘솔의 **시각적 확인**과 CLI의 **재현 가능한 명령**을 항상 교차 검증하며 진행합니다.

핵심 개념 및 구조

클라우드 서비스 3대 모델의 계층 구조

아래 계층이 위 계층의 기반을 제공하며, 위로 올라갈수록 사용자가 직접 관리할 영역이 줄어듭니다.



SaaS

완성된 소프트웨어를 브라우저와 API로 즉시 소비합니다

PaaS

인프라 구성 없이 애플리케이션 배포만 전담합니다

IaaS

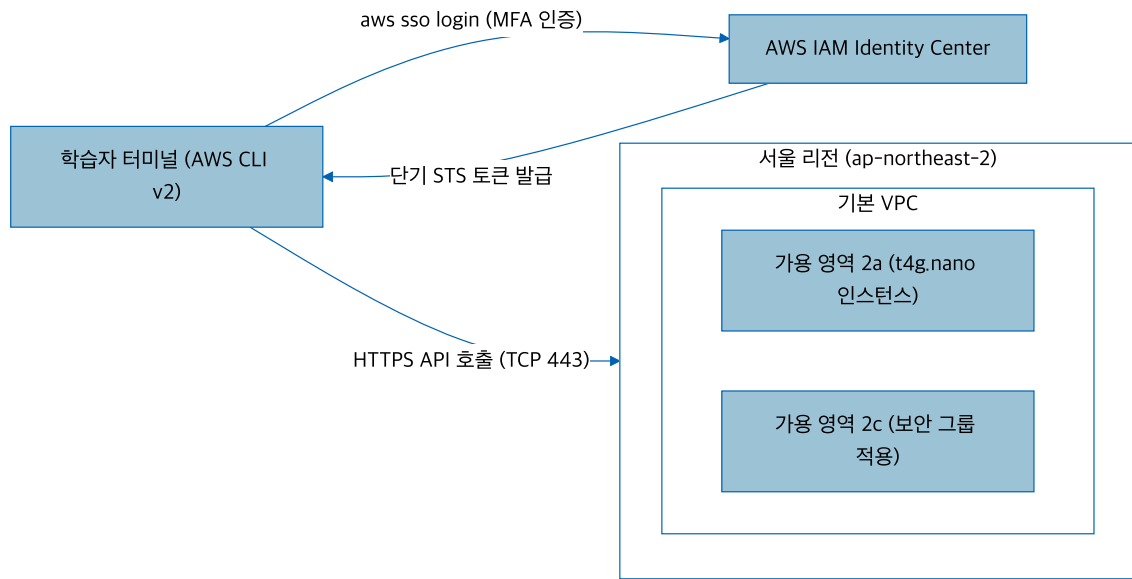
가상 서버와 사설망의 완전한 제어권을 보유합니다

서비스 모델별 관리 책임 경계

서비스 모델	관리 주체 (사용자 영역)	주요 특성	대표 서비스
SaaS	데이터 및 사용자 설정	완성된 소프트웨어를 즉시 소비	Notion, Slack, GitHub
PaaS	소스코드, 이미지, DB 스키마	애플리케이션 배포만 전담	Render, Railway, Aiven
IaaS	OS, 미들웨어, 네트워크, 런타임	가상 서버와 사설망 완전 제어	AWS, GCP, Azure

엔터프라이즈 환경이 **IaaS**를 선택하는 이유는 세밀한 사설망 격리와 아키텍처 제어권 때문입니다. 관리 범위가 넓어지는 대신 보안 경계와 비용을 직접 설계할 수 있습니다.

AWS 글로벌 인프라와 단기 자격 증명 제어 흐름



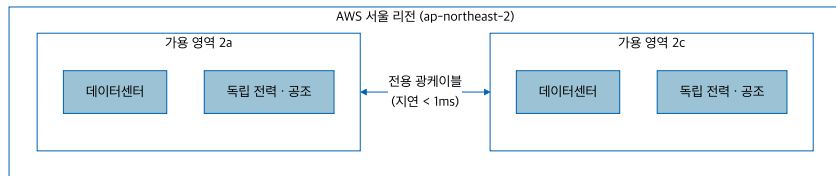
🔑 단기 자격 증명

브라우저 SSO와 MFA를 거쳐 유효 기간이 짧은 임시 토큰을 발급받습니다

🌐 리전 고정

실습 자원은 모두 서울 리전에 배치해 콘솔과 CLI의 조회 결과를 일치시킵니다

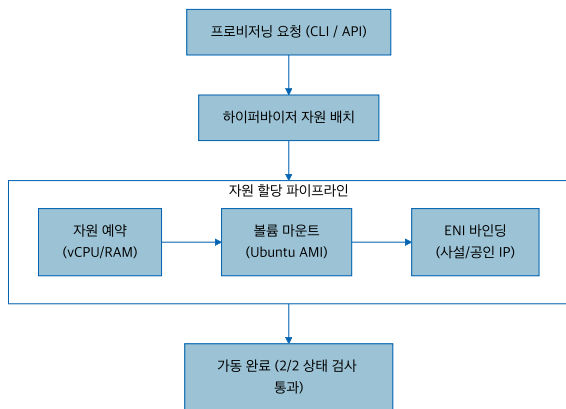
리전과 가용 영역의 물리 구조와 고가용성



인프라 단위	물리 구조 및 고가용성(HA) 역할
리전 (Region)	수천 km 격리된 지리 거점으로 광역 재난 차단 및 데이터 주권 준수
가용 영역 (AZ)	동일 리전 내 수십 km이격 센터군으로 단일 센터 장애 시 타 AZ 자동 절체

AZ 간 전용 광케이블로 **1ms 미만 초저지연 동기화**가 가능하며, 2개 이상 AZ에 분산 배치(Multi-AZ)하여 무중단 서비스를 보장합니다.

인프라 프로비저닝의 개념과 클라우드 패러다임



계층	대상 자원 및 실행 방식
인프라	VPC · 서브넷 · 보안 그룹 등 가상 사설망 및 방화벽 규칙 정의
서버	EC2 규격(<code>t4g.nano</code>) · AMI · EBS 볼륨 생성 및 SSH 키 주입
소프트웨어	원격 세션 접속 후 Docker 엔진 설치 및 컨테이너 배포

온프레미스에서 물리 서버 발주·랙 장착에 수 주 걸리던 구축을, 클라우드는 소프트웨어 정의(API)로 수십 초 내 자동 프로비저닝합니다.

보안 그룹의 상태 저장 방화벽 원리

검증 항목	인바운드 (Inbound)	아웃바운드 (Outbound)
기본 정책	모든 트래픽 차단 (화이트리스트 방식)	모든 외부 송출 트래픽 기본 허용
허용 규칙	지정한 포트/CIDR만 명시적 개방	필요한 외부 통신 대역 제어
상태 저장	인바운드로 허용된 연결의 반환 트래픽은 아웃바운드 규칙과 무관하게 자동 허용	아웃바운드로 시작된 요청의 수신 응답 트래픽 자동 허용

보안 그룹은 **상태 저장(Stateful)** 방식이므로 클라이언트의 요청 연결을 추적합니다. 80 포트로 웹 요청이 들어왔을 때 서버가 클라이언트에게 응답을 돌려주기 위한 임시 포트를 아웃바운드에 별도로 열 필요가 없습니다.

실습 단계

실습 로드맵: 포털 로그인부터 인스턴스 중지까지

구간	주제	이 구간에서 직접 확인하는 것
1	계정 배정과 액세스 포털 첫 로그인	일회용 암호로 로그인한 뒤 콘솔 진입
2	CLI 미인증 상태와 계정 가드레일	자격 증명 없는 터미널의 오류 메시지
3	비밀번호 변경과 MFA 등록	재로그인 시 요구되는 일회용 6자리 코드
4	AWS CLI v2 설치와 SSO 프로파일 연동	<code>aws-cli/2.</code> 버전 문자열과 세션 환경변수
5	역할 세션과 가용 영역 교차 검증	CLI 출력과 콘솔 화면의 일치 여부
6	보안 그룹과 SSH 키 페어 준비	인바운드 규칙 3종과 개인키 권한
7	<code>t4g.nano</code> 프로비저닝과 SSH 접속	<code>aarch64</code> 아키텍처와 중지 후 과금 상태

앞 구간은 **인증 체계를 세우고**, 뒤 구간은 그 자격 증명으로 **실제 인프라 자원을 생성하고 회수**합니다.

실습 계정 배치와 액세스 포털 첫 로그인

본인이 사용할 이메일 주소를 제출하면 그 주소로 전용 AWS 계정과 포털 사용자가 발급됩니다.

1. 발급 시 받는 항목: **포털 URL**, **사용자명**(`student01` 등), **일회용 암호**(발급 후 7일간 유효)
2. 포털 로그인 후 첫 화면의 **AWS 계정** 목록에서 배정된 계정 행을 클릭
3. 펼쳐진 권한 세트(`InfraTrainingPowerUser`) 우측의 **Management console** 링크로 콘솔 진입

<https://infra-lab-ai.awsapps.com/start>

화면	볼 위치	확인 항목
포털 첫 화면	<code>AWS 계정</code> 목록	배정 계정 1건과 권한 세트 이름
콘솔 상단 바	우측 리전 선택기	<code>아시아 태평양(서울) ap-northeast-2</code>
콘솔 상단 바	우측 계정 메뉴	계정 ID와 현재 로그인된 역할 이름

터미널 미인증 상태 확인과 계정 가드레일 점검

호스트 터미널에 아직 자격 증명이 없는 상태를 먼저 관찰합니다.

```
aws sts get-caller-identity
```

- 프로필이 없으면 `Unable to locate credentials`, 세션이 만료됐으면 `Error loading SS0 Token`이 출력됩니다

가드레일 항목	통제 내용
허용 인스턴스 유형	<code>t4g.nano</code> , <code>t4g.micro</code> , <code>t4g.small</code> 만 프로비저닝 가능
차단 방식	그 외 고비용 유형은 AWS API 레벨에서 즉시 <code>Deny</code> 응답
적용 지점	IAM Policy 또는 Organizations SCP의 조건부 거부문

가드레일은 콘솔 화면의 안내가 아니라 **API 레벨의 거부 정책**입니다. 콘솔에서 선택 자체는 가능해 보여도 생성 요청이 거부됩니다.

초기 비밀번호 변경과 가상 MFA 등록

비밀번호 단일 인증은 탈취되는 순간 곧바로 침입으로 이어지므로 2차 인증 수단을 함께 등록합니다.

1. 일회용 암호로 로그인하면 **비밀번호 변경 화면**이 먼저 표시되므로 새 비밀번호를 설정
2. 포털 우측 상단 **사용자 이름 메뉴**에서 MFA 디바이스 등록으로 이동
3. **인증 앱**을 선택하고 QR 코드를 스캔한 뒤 6자리 코드 입력
4. 로그아웃 후 새 비밀번호와 MFA 코드로 재로그인되는지 검증

인증 수단	목적	보안 이점
새 비밀번호	일회용 암호 폐기	1차 지식 기반 인증 강화
가상 MFA	일회용 TOTP 토큰 발급	비밀번호 탈취만으로는 침입 불가

최신 AWS CLI v2 설치와 터미널 환경 확인

```
# 버전 확인 (v2 최신 버전 전제)
command -v aws
aws --version

# macOS / Linux / WSL2 설치
curl -fsSL https://awscli.amazonaws.com/v2/install.sh | bash
export PATH="$HOME/.local/bin:$PATH"

# Windows (Git Bash) 설치
curl -o AWSCLIV2.msi https://awscli.amazonaws.com/AWSCLIV2.msi
MSYS_NO_PATHCONV=1 msixexec.exe /i AWSCLIV2.msi /qn
```

환경	터미널	실습 호환성 및 권장
macOS / Linux	<code>zsh</code> / <code>bash</code>	표준 POSIX 셸 (완벽 호환)
Windows	<code>Git Bash</code>	실습 표준 (<code>export</code> , <code>chmod</code> 동작)
Windows	<code>PowerShell</code>	호출 동일, 키 권한은 <code>icaccls</code> 로 조정

본 과정은 브라우저 SSO 및 토큰 자동 갱신을 지원하는 **AWS CLI v2 최신**을 기준으로 합니다. Windows 사용자는 `Git Bash` 나 `WSL2`를 쓰면 Linux와 동일한 명령어로 실습할 수 있습니다.

SSO 프로필 등록과 CLI 세션 환경변수 고정

```
# 대화형 SSO 프로필 등록
aws configure sso --profile student01

# SSO session name: infra-training
# SSO start URL: 배정받은_포털_URL
# SSO region: ap-northeast-2
# SSO registration scopes: sso:account:access
# (승인 후 계정/권한 세트를 목록에서 선택)
# CLI default client Region: ap-northeast-2
# CLI default output format: json
# CLI profile name: student01

# 브라우저 인증 승인 후 환경변수 고정
aws sso login --profile student01
export AWS_PROFILE="student01"
export AWS_REGION="ap-northeast-2"
export AWS_PAGER=""
```

환경변수	역할
<code>AWS_PROFILE</code>	모든 명령에 프로필 옵션 자동 적용
<code>AWS_REGION</code>	대상 리전을 서울로 고정
<code>AWS_PAGER</code>	빈 문자열 지정으로 페이저 정지 해제

브라우저에서 터미널의 8자리 코드와 화면의 코드가 같은지 확인하고 **Allow access**를 선택해야 토큰이 발급됩니다. 승인 전까지 터미널은 대기 상태를 유지합니다.

역할 세션과 가용 영역 교차 검증

```
# 현재 세션의 IAM 역할 자격 증명 확인
```

```
aws sts get-caller-identity
```

```
# 서울 리전 내 가용 영역 목록 조회
```

```
aws ec2 describe-availability-zones \
  --filters "Name=state,Values=available" \
  --query "AvailabilityZones[0].{Zone:ZoneName,State:State}" \
  --output table
```

확인 위치

정상 판정 기준

CLI **Arn**

assumed-role 형태의 역할 세션

CLI 가용 영역

2a, **2b**, **2c**, **2d** 4개 조회

VPC 콘솔 → **서브넷**

기본 서브넷 4개의 **가용 영역** 열이 동일

터미널 출력과 콘솔 화면이 같은 값을 가리키는지 대조하는 것이 이 구간의 목적입니다. 기본 VPC에는 가용 영역마다 서브넷이 하나씩 자동 생성되어 있습니다.

기본 VPC 조회와 실습용 보안 그룹 생성

```
# 서울 리전의 기본 VPC ID 조회
export VPC_ID=$(aws ec2 describe-vpcs \
  --filters "Name=is-default,Values=true" \
  --query "Vpcs[0].VpcId" --output text)
echo "기본 VPC ID: $VPC_ID"

# 개인 실습용 보안 그룹 생성
export MY_SG_NAME="student01-web-sg"
export MY_SG_ID=$(aws ec2 create-security-group \
  --group-name "$MY_SG_NAME" \
  --description "SG for Spring Boot practice" \
  --vpc-id "$VPC_ID" \
  --query "GroupId" --output text)
echo "생성된 보안 그룹 ID: $MY_SG_ID"
```

파라미터	설명
<code>--group-name</code>	개인별로 식별 가능한 보안 그룹 이름
<code>--vpc-id</code>	인스턴스가 소속될 기본 사설망 ID
<code>--query</code>	응답 JSON에서 필요한 필드만 추출

실습 리소스에는 생성 시점에 `Name` 과 `Course=infra-training` 태그를 부여해 비용 추적 단위를 통일합니다.

보안 그룹 인바운드 규칙 개방

```
# 내 호스트 공인 IP 자동 감지
export MY_IP=$(curl -fsS https://checkip.amazonaws.com)
echo "내 공인 IP: $MY_IP"

# SSH는 내 IP만, HTTP와 App 포트는 전역 허용
aws ec2 authorize-security-group-ingress \
  --group-id "$MY_SG_ID" \
  --protocol tcp --port 22 --cidr "$MY_IP/32"

aws ec2 authorize-security-group-ingress \
  --group-id "$MY_SG_ID" \
  --protocol tcp --port 80 --cidr 0.0.0.0/0

aws ec2 authorize-security-group-ingress \
  --group-id "$MY_SG_ID" \
  --protocol tcp --port 8080 --cidr 0.0.0.0/0
```

포트	허용 범위	용도
22	내 공인 IP 단일 주소	SSH 원격 접속
80	0.0.0.0/0	HTTP 웹 서비스 공개
8080	0.0.0.0/0	애플리케이션 직접 노출

SSH 포트를 전역 개방하면 자동화된 무차별 대입 시도의 표적이 됩니다. 접속 출발지 IP가 바뀌면 규칙을 다시 등록합니다.

등록된 인바운드 규칙의 콘솔 교차 검증

CLI 변경은 콘솔에 즉시 반영됩니다. 같은 자원을 두 경로에서 확인해 조작 결과를 확정합니다.

```
aws ec2 describe-security-groups \
  --group-ids "$MY_SG_ID" \
  --query "SecurityGroups[0].IpPermissions" \
  --output table
```

1. EC2 콘솔 좌측 메뉴 **네트워크 및 보안** → **보안 그룹**에서 생성한 그룹 이름을 클릭
2. 하단 **인바운드 규칙** 탭의 **포트 범위**와 **소스** 열에서 22, 80, 8080을 확인

검증 경로	확인 내용
CLI 테이블 출력	세 개의 IpPermissions 항목이 표 형태로 조회
콘솔 인바운드 규칙 탭	22번의 소스가 내 공인 IP /32 로 한정

SSH 키 페어 발급과 개인키 권한 축소

```
# 로컬 SSH 디렉터리 준비 및 키 페어 생성
mkdir -p ~/.ssh
export MY_KEY_NAME="student01-key"

aws ec2 create-key-pair \
  --key-name "$MY_KEY_NAME" \
  --query "KeyMaterial" \
  --output text > ~/.ssh/"$MY_KEY_NAME".pem

# 개인키를 소유자 읽기 전용으로 축소
chmod 400 ~/.ssh/"$MY_KEY_NAME".pem
ls -l ~/.ssh/"$MY_KEY_NAME".pem
```

설정 요소	설명
KeyMaterial	로컬에만 보관되는 비대칭 개인키 텍스트
chmod 400	소유자 읽기 전용 (Git Bash · WSL2 지원)

개인키는 생성 시점에 **단 한 번만** 반환됩니다. Windows 환경에서는 **Git Bash** 나 **WSL2**를 사용하면 **chmod 400** 이 그대로 동작해 권한 오류를 방지할 수 있습니다.

최신 Ubuntu ARM AMI 조회와 t4g.nano 프로비저닝

```
export AMI_PARAM="/aws/service/canonical/ubuntu/server/26.04/stable/current/arm64/hvm/ebs-gp3/ami-id"
export AMI_ID=$(aws ssm get-parameter --name "$AMI_PARAM" --query "Parameter.Value" --output text)

export INSTANCE_ID=$(aws ec2 run-instances \
  --image-id "$AMI_ID" --instance-type t4g.nano \
  --key-name "$MY_KEY_NAME" --security-group-ids "$MY_SG_ID" \
  --query "Instances[0].InstanceId" --output text)
```

명령어	역할
<code>aws ssm get-parameter</code>	Parameter Store에서 공식 AMI ID를 동적으로 조회
<code>aws ec2 run-instances</code>	지정한 규격과 보안 그룹으로 가상 서버 시작

AMI ID는 리전마다 다르고 주기적으로 갱신됩니다. 값을 문서에 적어두지 않고 **조회 명령으로 매번 획득**하는 것이 표준 방식입니다.

실행 상태 대기과 퍼블릭 IP 조회

프로비저닝은 비동기로 진행되므로 임의의 대기 대신 공식 대기 명령을 사용합니다.

```
aws ec2 wait instance-running --instance-ids "$INSTANCE_ID"
aws ec2 wait instance-status-ok --instance-ids "$INSTANCE_ID"

export PUBLIC_IP=$(aws ec2 describe-instances \
  --instance-ids "$INSTANCE_ID" \
  --query "Reservations[0].Instances[0].PublicIpAddress" --output text)
echo "할당된 퍼블릭 IP: $PUBLIC_IP"
```

대기 명령	완료 판정 시점	콘솔에서 볼 위치
<code>wait instance-running</code>	수명 주기가 <code>running</code> 으로 전환	<code>인스턴스</code> 목록의 <code>인스턴스 상태</code> 열
<code>wait instance-status-ok</code>	시스템·인스턴스 검사 모두 통과	같은 목록의 <code>상태 검사</code> 열 <code>2/2개 검사 통과</code>

상태 검사 통과까지 대략 2분이 걸립니다. 퍼블릭 IP는 인스턴스를 선택했을 때 하단 **세부 정보** 탭의 `퍼블릭 IPv4 주소`에서도 같은 값을 확인할 수 있습니다.

SSH 원격 접속과 aarch64 아키텍처 검증

개인키 기반 접속은 비밀번호를 네트워크로 전송하지 않으므로 무차별 대입 시도에 노출되지 않습니다.

```
ssh -i ~/.ssh/"$MY_KEY_NAME".pem -o StrictHostKeyChecking=accept-new ubuntu@"$PUBLIC_IP"
```

```
uname -m           # 출력: aarch64
cat /etc/os-release | grep "PRETTY_NAME" # Ubuntu 배포판 확인
exit               # 원격 세션 종료
```

명령어	확인 내용
<code>uname -m</code>	<code>aarch64</code> 출력으로 ARM Graviton 계열임을 확인
<code>grep PRETTY_NAME</code>	원격 인스턴스의 리눅스 배포판과 버전 확인

로컬에서 빌드한 이미지의 아키텍처가 다르면 이 인스턴스에서 실행되지 않습니다. 크로스 플랫폼 빌드가 필요한 이유가 여기에 있습니다.

인스턴스 중지과 과금 상태 비교

실습을 마치면 즉시 인스턴스를 중지해 컴퓨팅 요금을 동결합니다.

```
aws ec2 stop-instances --instance-ids "$INSTANCE_ID"
aws ec2 wait instance-stopped --instance-ids "$INSTANCE_ID"
```

상태	컴퓨팅 요금	스토리지 요금	특징
Running	시간당 과금	부과	인스턴스 동작 중
Stopped	면제	EBS 보관료 소액	다음 실습에서 재사용 가능
Terminated	없음	없음	인스턴스와 루트 볼륨 완전 소멸

후속 실습에서 이 인스턴스를 다시 사용하므로 **terminate**가 아니라 **stop**을 실행합니다. 중지과 동시에 동적 공인 IP가 회수되어 콘솔 퍼블릭 IPv4 주소 칸이 비워지며, 재시작 시 새 주소가 할당됩니다.

Appendix

AWS 프리 티어 정책 개편

기존의 12개월 무료 방식에서 최대 6개월 또는 최대 \$200 크레딧을 제공하는 무료 플랜으로 변경되었습니다.

구분	과거 방식	신규 방식
제공 형태	12개월간 서비스별 무료 한도	가입 크레딧 \$100 + 온보딩 완료 시 \$100
한도 초과 시	신용카드로 즉시 과금	서비스 자동 일시 정지로 과금 차단
종료 조건	12개월 경과	6개월 경과 또는 크레딧 소진

정지 후 일정 기간 내에 유료 플랜으로 전환하지 않으면 리소스가 영구 삭제됩니다. 본 과정이 개인 신용카드 가입 대신 사전 배포 계정을 사용하는 이유입니다.

인스턴스 유형 강제 가드레일의 동작 원리

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowOnlyT4gNanoMicroSmall",
      "Effect": "Deny",
      "Action": "ec2:RunInstances",
      "Resource": "arn:aws:ec2:*:*:instance/*",
      "Condition": {
        "StringNotLike": {
          "ec2:InstanceType": [
            "t4g.nano", "t4g.micro", "t4g.small"
          ]
        }
      }
    }
  ]
}
```

정책 요소	역할
Effect: Deny	명시적 거부는 어떤 허용보다 우선 적용
Condition	요청 파라미터가 조건에 맞을 때만 규칙 발동
StringNotLike	허용 목록에 없는 유형이면 조건 성립

허용 목록 외 인스턴스는 **UnauthorizedOperation**으로 거부됩니다. AMI와 다른 아키텍처의 타입은 정책 평가 전 **InvalidParameterValue**로 먼저 실패합니다.

교육 환경의 실시간 비용 통제 아키텍처

수업 종료 후 남겨진 인스턴스가 야간 내내 과금되는 상황을 자동화로 차단합니다.

🕒 야간 일괄 자동 중지

Amazon EventBridge의 Cron 스케줄러가 정해진 시각에 Lambda를 호출해 미중지 인스턴스를 일괄 **stop** 처리합니다

🕒 유휴 인스턴스 자동 중지

CloudWatch Alarm이 CPU 사용률 저조 상태가 일정 시간 지속되면 인스턴스를 스스로 정지시킵니다

통제 수단	발동 조건	처리 범위
EventBridge 스케줄	지정한 시각 도달	미중지 인스턴스를 일괄 stop
CloudWatch Alarm	CPU 사용률 임계 미만 지속	유휴 인스턴스를 개별 정지

비용 통제는 사용자의 주의력이 아니라 **자동화된 정책**으로 설계합니다. 사람이 잊어버릴 수 있다는 전제에서 출발하는 것이 클라우드 운영의 기본입니다.

학습용 가성비 대안 인프라

서비스	특징
AWS Lightsail	VPC와 보안 그룹 설정 없이 가상 서버·고정 IP·스토리지를 월 정액제로 제공
Vultr	서울을 포함한 글로벌 데이터센터를 시간·월 단위 저가 요금제로 제공
Google Cloud Free Tier	신규 가입 시 90일 크레딧과 특정 규격의 상시 무료 사용량 지원
Oracle Cloud Free Tier	고성능 ARM 인스턴스를 상시 무료 범위로 제공

개인 학습과 사이드 프로젝트에는 위 대안이 경제적입니다. 다만 VPC 격리와 IAM 권한 설계 같은 **엔터프라이즈 아키텍처 학습**은 IaaS 환경에서만 가능합니다.

글로벌 클라우드 벤더 대응 서비스 비교

구분	AWS	Microsoft Azure	Google Cloud
가상 서버	EC2	Virtual Machines	Compute Engine
객체 스토리지	S3	Blob Storage	Cloud Storage
관리형 관계형 DB	RDS	Azure Database	Cloud SQL
관리형 Kubernetes	EKS	AKS	GKE

제품명은 벤더마다 다르지만 가상 서버, 객체 스토리지, 관리형 DB, 컨테이너 오케스트레이션이라는 구성 요소의 역할은 동일합니다. 한 벤더의 구조를 이해하면 다른 벤더로의 전환 비용이 크게 줄어듭니다.

CIDR 표기법과 IP 주소 대역 설계

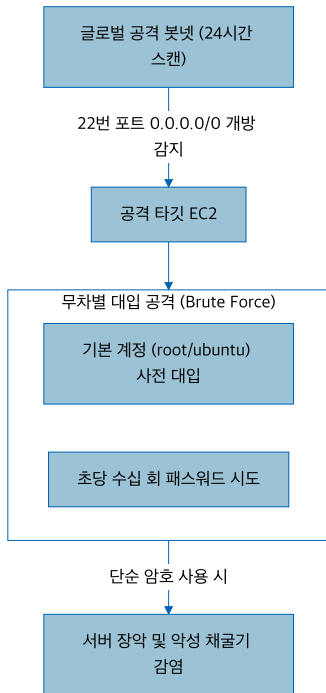
CIDR 표기	할당 가능 IP 개수	용도 및 네트워크 의미
<code>0.0.0.0/0</code>	약 43억 개 (2^{32} , 전체)	모든 IPv4 인터넷 주소 (웹 80·443 포트 전체 개방)
<code>172.31.0.0/16</code>	65,536개 (2^{16})	AWS 기본 VPC 사설 네트워크 대역
<code>172.31.0.0/20</code>	4,096개 (2^{12})	가용 영역(AZ)별 기본 서브넷 분할 대역
<code>내 IP/32</code>	단 1개 (2^0)	단일 특정 호스트 (SSH 22번 관리 포트 허용 필수)

🔗 CIDR (사이드어) 표기 원리

`IP/N` 표기에서 앞 \$N\$비트는 네트워크 주소로 고정하고 나머지 \$32-N\$비트를 호스트에 유연하게 할당합니다.

`0.0.0.0/0`은 모든 사용자에게 열려 있다는 뜻입니다.
SSH(22)나 DB(3306)를 전체 개방하면 즉시 침해 표적이
되므로 관리 포트는 반드시 `/32`로 제한합니다.

SSH 무차별 대입 공격의 위협과 방어 체계



방어 계층	방어 기법	핵심 동작 및 기대 효과
네트워크	보안 그룹 내 IP/32	인가되지 않은 IP의 22번 포트 원천 차단
인증	비대칭 키 페어 강제	비밀번호 로그인 비활성화 및 PEM 서명 필수
모니터링	fail2ban · GuardDuty	비정상 실패 IP 자동 차단 및 실시간 경보

공인 IP에 SSH가 전체 개방되면 수 분 내에 봇넷의 사전식 대입 공격이 시작됩니다. **CIDR 제한**과 **개인키 인증**의 다층 방어가 필수적입니다.

Windows 호스트의 SSH 키 권한과 PuTTY 계열 도구

```
# PowerShell: 상속 권한 제거 후 소유자만 읽기 허용
$KEY="$env:USERPROFILE\.ssh\student-key.pem"
icacls $KEY /inheritance:r
icacls $KEY /grant:r "$env:USERNAME:R"
ssh -i $KEY ubuntu@<인스턴스_공인_IP>
```

```
# PuTTY 전용 키 형식으로 변환 (.pem → .ppk)
puttygen student-key.pem -o student-key.ppk
```

환경	사용하는 키 파일
Git Bash · WSL2	<code>.pem</code> (<code>chmod 400</code> 적용)
PuTTY 계열	<code>.ppk</code> (PuTTYgen 변환본)

도구	역할
PuTTYgen	<code>.pem</code> 과 <code>.ppk</code> 상호 변환
PuTTY	단일 SSH 세션 접속
Pageant	메모리 상주 키 에이전트
SuperPuTTY	PuTTY 세션 탭 관리 및 목록 저장

실습 표준은 `Git Bash`이며 `chmod 400`이 그대로 적용됩니다. PowerShell은 권한 상속이 남으면 `UNPROTECTED PRIVATE KEY FILE` 오류로 거부하고, PuTTY는 `.pem`을 직접 읽지 못합니다.

핵심 기술 면접 질문

단기 자격 증명

IAM 영구 액세스 키 대신 IAM Identity Center 단기 자격 증명을 권장하는 이유는 무엇인가요?

상태 저장 방화벽

AWS 보안 그룹이 상태 저장(Stateful) 방화벽이라는 것은 어떤 의미인가요?

리전과 가용 영역

클라우드 고가용성 아키텍처에서 리전과 가용 영역의 차이는 무엇인가요?

IaaS와 PaaS의 선택 기준

클라우드 서비스 모델 중 IaaS와 PaaS의 차이점과 선택 기준은 무엇인가요?

핵심 기술 면접 질문

CIDR 표기법과 위험성

보안 그룹에서 0.0.0.0/0과 내 IP/32의 차이점 및 SSH 전체 개방의 위험성은 무엇인가요?

무차별 대입 공격 방어

SSH 무차별 대입 공격(Brute Force)을 방어하기 위한 3단계 핵심 대책은 무엇인가요?

모범 답변

단기 자격 증명을 권장하는 이유

영구 액세스 키는 발급 후 개발자 PC나 설정 파일에 평문으로 계속 남아 있어, 저장소에 실수로 커밋되거나 악성코드에 노출되면 즉시 침해와 과금 사고로 이어집니다. IAM Identity Center 기반의 단기 자격 증명은 브라우저 SSO와 MFA를 거쳐 수 시간만 유효한 임시 STS 토큰을 발급받으므로, 유출되더라도 곧 만료되어 공격 표면이 최소화됩니다.

상대 저장 방화벽의 의미

인바운드로 들어온 요청 트래픽의 연결 상태를 추적해, 해당 요청에 대한 응답 아웃바운드 트래픽을 아웃바운드 규칙과 무관하게 자동 허용한다는 뜻입니다. 외부에서 80 포트로 들어온 요청에 서버가 응답할 때 임시 포트의 아웃바운드 규칙을 따로 열지 않아도 통신이 성립합니다.

리전과 가용 영역의 차이

리전은 지리적으로 분리된 거점으로, 통신 지연과 규제 준수를 고려해 선택합니다. 가용 영역은 동일 리전 안에서 일정 거리를 두고 독립된 전력·냉각·전용 광케이블망을 갖춘 물리적 데이터센터 건물입니다.고가용성 설계에서는 단일 리전 내 최소 2개 이상의 가용 영역에 EC2나 RDS를 다중화해, 한 데이터센터가 멈춰도 서비스가 지속되게 합니다.

모범 답변

IaaS와 PaaS의 선택 기준

PaaS는 런타임과 배포 파이프라인을 플랫폼이 책임지므로 소스코드와 스키마에만 집중할 수 있어 초기 속도가 빠릅니다. 반면 사설망 격리, 보안 그룹 설계, 인스턴스 규격 제어가 요구되는 엔터프라이즈 환경에서는 IaaS를 선택해 OS와 네트워크 계층까지 직접 관리합니다.

CIDR 표기법과 SSH 0.0.0.0/0 개방의 위험성

CIDR의 슬래시 뒤 숫자는 서브넷 마스크 비트 수입니다. `0.0.0.0/0`은 0비트만 고정되어 전 세계 모든 IPv4 인터넷 주소를 뜻하며, `/32`는 32비트 전체가 고정되어 단 1개의 특정 호스트만을 의미합니다. SSH(22) 포트를 `0.0.0.0/0`으로 개방하면 전 세계의 자동화 공격 봇넷으로부터 무차별 대입 공격의 표적이 되므로, 관리 포트는 반드시 관리자의 단일 공인 IP인 `/32`로 엄격히 제한해야 합니다.

SSH 무차별 대입 공격의 3단계 방어 대책

첫째, 네트워크 계층에서 보안 그룹 인바운드를 `내 IP/32`로 제한해 공격 유입 자체를 차단합니다. 둘째, 인증 계층에서 패스워드 로그인을 끄고(`PasswordAuthentication no`) 비대칭 개인키(`.pem`) 인증만 강제합니다. 셋째, 호스트 계층에서 실패 반복 IP를 자동 차단하는 `fail2ban`과 비정상 스캔을 감지하는 `Amazon GuardDuty`를 연계해 다층 방어를 완성합니다.

깊이 알아보기

🔑 IAM Identity Center

<https://docs.aws.amazon.com/singlesignon/latest/userguide/what-is.html>

📖 AWS CLI v2 설치

<https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>

🔑 SSO 프로필 구성

<https://docs.aws.amazon.com/cli/latest/userguide/sso-configure-profile-token.html>

📖 EC2 보안 그룹

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html>

깊이 알아보기

🔗 리전과 가용 영역

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-regions-availability-zones.html>

KEY EC2 키 페어

<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-key-pairs.html>



- EOF -